Listing 13-3

Command.java

```
public interface Command
{
  public void execute() throws Exception;
}
```

This may not seem very impressive. But imagine what would happen if one of the `Command` objects in the linked list cloned itself and then put the clone back on the list. The list would never go empty, and the `run()` function would never return.

Consider the test case in Listing 13-4. It creates something called a `SleepCommand`. Among other things, it passes a delay of 1000 ms to the constructor of the `SleepCommand`. It then puts the `SleepCommand` into the `ActiveObjectEngine`. After calling `run()`, it expects that a certain number of milliseconds has elapsed.

Listing 13-4

TestSleepCommand.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class TestSleepCommand extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"TestSleepCommand"});
  }

  public TestSleepCommand(String name)
  {
    super(name);
  }

  private boolean commandExecuted = false;

  public void testSleep() throws Exception
  {
    Command wakeup = new Command()
    {
      public void execute() {commandExecuted = true;}
    };
    ActiveObjectEngine e = new ActiveObjectEngine();
    SleepCommand c = new SleepCommand(1000,e,wakeup);
    e.addCommand(c);
    long start = System.currentTimeMillis();
    e.run();
    long stop = System.currentTimeMillis();
    long sleepTime = (stop-start);
    assert("SleepTime " + sleepTime + " expected > 1000",
           sleepTime > 1000);
    assert("SleepTime " + sleepTime + " expected < 1100",
            sleepTime < 1100);
    assert("Command Executed", commandExecuted);
  }
}
```

Let's look at this test case more closely. The constructor of the `SleepCommand` contains three arguments. The first is the delay time in milliseconds. The second is the `ActiveObjectEngine` that the command will be running in. Finally, there is another command object called `wakeup`. The intent is that the `SleepCommand` will wait for the specified number of milliseconds and will then execute the `wakeup` command.

Listing 13-5 shows the implementation of `SleepCommand`. On execution, `SleepCommand` checks to see if it has been executed previously. If not, then it records the start time. If the delay time has not passed, it puts itself back in the `ActiveObjectEngine`. If the delay time has passed, it puts the `wakeup` command into the `ActiveObjectEngine`.

Listing 13-5

`SleepCommand.java`

```
public class SleepCommand implements Command
{
  private Command wakeupCommand = null;
  private ActiveObjectEngine engine = null;
  private long sleepTime = 0;
  private long startTime = 0;
  private boolean started = false;

  public SleepCommand(long milliseconds, ActiveObjectEngine e,
                      Command wakeupCommand)
  {
    sleepTime = milliseconds;
    engine = e;
    this.wakeupCommand = wakeupCommand;
  }

  public void execute() throws Exception
  {
    long currentTime = System.currentTimeMillis();
    if (!started)
    {
      started = true;
      startTime = currentTime;
      engine.addCommand(this);
    }
    else if ((currentTime - startTime) < sleepTime)
    {
      engine.addCommand(this);
    }
    else
    {
      engine.addCommand(wakeupCommand);
    }
  }
}
```

We can draw an analogy between this program and a multithreaded program that is waiting for an event. When a thread in a multithreaded program waits for an event, it usually invokes some operating system call that blocks the thread until the event has occurred. The program in Listing 13-5 does not block. Instead, if the event it is waiting for `((currentTime - startTime) < sleepTime)` has not occurred, it simply puts itself back into the `ActiveObjectEngine`.

Building multithreaded systems using variations of this technique has been, and will continue to be, a very common practice. Threads of this kind have been known as *run-to-completion* tasks (RTC), because each `Command` instance runs to completion before the next `Command` instance can run. The name RTC implies that the `Command` instances do not block.

The fact that the `Command` instances all run to completion gives RTC threads the interesting advantage that they all share the same run-time stack. Unlike the threads in a traditional multithreaded system, it is not necessary to define or allocate a separate run-time stack for each RTC thread. This can be a powerful advantage in memory-constrained systems with many threads.

Continuing our example, Listing 13-6 shows a simple program that makes use of `SleepCommand` and exhibits multithreaded behavior. This program is called `DelayedTyper`.

Listing 13-6

DelayedTyper.java

```
public class DelayedTyper implements Command
{
  private long itsDelay;
  private char itsChar;
  private static ActiveObjectEngine engine =
    new ActiveObjectEngine();
  private static boolean stop = false;

  public static void main(String args[])
  {
    engine.addCommand(new DelayedTyper(100,'1'));
    engine.addCommand(new DelayedTyper(300,'3'));
    engine.addCommand(new DelayedTyper(500,'5'));
    engine.addCommand(new DelayedTyper(700,'7'));

    Command stopCommand = new Command()
    {
      public void execute() {stop=true;}
    };

    engine.addCommand(
      new SleepCommand(20000,engine,stopCommand));
    engine.run();
  }

  public DelayedTyper(long delay, char c)
  {
    itsDelay = delay;
    itsChar = c;
  }

  public void execute() throws Exception
  {
    System.out.print(itsChar);
    if (!stop)
      delayAndRepeat();
  }
```

```
  private void delayAndRepeat() throws Exception
  {
    engine.addCommand(new SleepCommand(itsDelay,engine,this);
  }
}
```

Notice that `DelayedTyper` implements `Command`. The `execute` method simply prints a character that was passed at construction, checks the `stop` flag, and, if not set, invokes `delayAndRepeat`. The `delayAndRepeat` method constructs a `SleepCommand`, using the delay that was passed in at construction. It then inserts the `SleepCommand` into the `ActiveObjectEngine`.

The behavior of this `Command` is easy to predict. In effect, it hangs in a loop repeatedly typing a specified character and waiting for a specified delay. It exits the loop when the `stop` flag is set.

The main program of `DelayedTyper` starts several `DelayedTyper` instances going in the `ActiveObjectEngine`, each with its own character and delay. It then invokes a `SleepCommand` that will set the `stop` flag after a while. Running this program produces a simple string of 1's, 3's, 5's and 7's. Running it again produces a similar, but different string. Here are two typical runs:

```
13571131151137111315113171513111315173111135111371153111357...
13571113151317113151131171351113115173111315113171135111317...
```

These strings are different because the CPU clock and the real-time clock aren't in perfect sync. This kind of nondeterministic behavior is the hallmark of multithreaded systems.

Nondeterministic behavior is also the source of much woe, anguish, and pain. As anyone who has worked on embedded real-time systems knows, it's tough to debug nondeterministic behavior.

Conclusion

The simplicity of the COMMAND pattern belies its versatility. COMMAND can be used for a wonderful variety of purposes ranging from database transactions, to device control, to multithreaded nuclei, to GUI do/undo administration.

It has been suggested that the COMMAND pattern breaks the OO paradigm because it emphasizes functions over classes. That may be true, but in the real world of the software developer, the COMMAND pattern can be very useful.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Lavender, R. G., and D. C. Schmidt. *Active Object: An Object Behavioral Pattern for Concurrent Programming*, in "Pattern Languages of Program Design" (J. O. Coplien, J. Vlissides, and N. Kerth, eds.). Reading, MA: Addison–Wesley, 1996.

14

TEMPLATE METHOD & STRATEGY: Inheritance vs. Delegation



"The best strategy in life is diligence."

—Chinese Proverb

Way, way back in the early 90s—back in the early days of OO—we were all quite taken with the notion of inheritance. The implications of the relationship were profound. With inheritance we could *program by difference*! That is, given some class that did something almost useful to us, we could create a subclass and change only the bits we didn't like. We could reuse code simply by inheriting it! We could establish whole taxonomies of software structures, each level of which reused code from the levels above. It was a brave new world.

Like most brave new worlds, this one turned out to be a bit too starry eyed. By 1995, it was clear that inheritance was very easy to overuse and that overuse of inheritance was very costly. Gamma, Helm, Johnson, and Vlissides went so far as to stress, "*Favor object composition over class inheritance.*"[1] So we cut back on our use of inheritance, often replacing it with composition or delegation.

This chapter is the story of two patterns that epitomize the difference between inheritance and delegation. TEMPLATE METHOD and STRATEGY solve similar problems and can often be used interchangeably. However, TEMPLATE METHOD uses inheritance to solve the problem, whereas STRATEGY uses delegation.

TEMPLATE METHOD and STRATEGY both solve the problem of separating a generic algorithm from a detailed context. We see the need for this very frequently in software design. We have an algorithm that is generically applicable. In order to conform to the Dependency-Inversion Principle (DIP), we want to make sure that the

1. [GOF95], p. 20.

generic algorithm does not depend on the detailed implementation. Rather, we want the generic algorithm and the detailed implementation to depend on abstractions.

TEMPLATE METHOD

Consider all the programs you have written. Many probably have this fundamental main-loop structure.

```
Initialize();
while (!done()) // main loop
{
  Idle();       // do something useful.
}
Cleanup();
```

First we initialize the application. Then we enter the main loop. In the main loop we do whatever the program needs us to do. We might process GUI events or perhaps process database records. Finally, once we are done, we exit the main loop and clean up before we exit.

This structure is so common that we can capture it in a class named `Application`. Then we can reuse that class for every new program we want to write. Think of it! We never have to write that loop again![2]

For example, consider Listing 14-1. Here we see all the elements of the standard program. The `InputStreamReader` and `BufferedReader` are initialized. There is a main loop that reads Fahrenheit readings from the `BufferedReader` and prints out Celsius conversions. At the end, an exit message is printed.

Listing 14-1

`ftoc raw`

```
import java.io.*;
public class ftocraw
{
  public static void main(String[] args) throws Exception
  {
    InputStreamReader isr = new InputStreamReader(System.in);
    BufferedReader br = new BufferedReader(isr);
    boolean done = false;
    while (!done)
    {
      String fahrString = br.readLine();
      if (fahrString == null || fahrString.length() == 0)
        done = true;
      else
      {
        double fahr = Double.parseDouble(fahrString);
        double celcius = 5.0/9.0*(fahr-32);
        System.out.println("F=" + fahr + ", C=" + celcius);
      }
    }
    System.out.println("ftoc exit");
  }
}
```

This program has all the elements of the main-loop structure. It does a little initialization, does its work in a main loop, and then cleans up and exits.

2. I've also got this bridge I'd like to sell you.

We can separate this fundamental structure from the `ftoc` program by employing the TEMPLATE METHOD pattern. This pattern places all the generic code into an implemented method of an abstract base class. The implemented method captures the generic algorithm, but defers all details to abstract methods of the base class.

So, for example, we can capture the main-loop structure in an abstract base class called `Application`. (See Listing 14-2.)

Listing 14-2

`Application.java`

```
public abstract class Application
{
  private boolean isDone = false;

  protected abstract void init();
  protected abstract void idle();
  protected abstract void cleanup();

  protected void setDone()
  {isDone = true;}

  protected boolean done()
  {return isDone;}

  public void run()
  {
    init();
    while (!done())
      idle();
    cleanup();
  }
}
```

This class describes a generic main-loop application. We can see the main loop in the implemented `run` function. We can also see that all the work is being deferred to the abstract methods `init`, `idle`, and `cleanup`. The `init` method takes care of any initialization we need done. The `idle` method does the main work of the program and will be called repeatedly until `setDone` is called. The `cleanup` method does whatever needs to be done before we exit.

We can rewrite the `ftoc` class by inheriting from `Application` and just filling in the abstract methods. Listing 14-3 show what this looks like.

Listing 14-3

`ftocTemplateMethod.java`

```
import java.io.*;
public class ftocTemplateMethod extends Application
{
  private InputStreamReader isr;
  private BufferedReader br;

  public static void main(String[] args) throws Exception
  {
    (new ftocTemplateMethod()).run();
  }
```

```
  protected void init()
  {
    isr = new InputStreamReader(System.in);
    br = new BufferedReader(isr);
  }

  protected void idle()
  {
    String fahrString = readLineAndReturnNullIfError();
    if (fahrString == null || fahrString.length() == 0)
      setDone();
    else
    {
      double fahr = Double.parseDouble(fahrString);
      double celcius = 5.0/9.0*(fahr-32);
      System.out.println("F=" + fahr + ", C=" + celcius);
    }
  }

  protected void cleanup()
  {
    System.out.println("ftoc exit");
  }

  private String readLineAndReturnNullIfError()
  {
    String s;
    try
    {
      s = br.readLine();
    }
    catch(IOException e)
    {
      s = null;
    }
    return s;
  }
}
```

Dealing with the exception made the code get a little longer, but it's easy to see how the old `ftoc` application has been fit into the TEMPLATE METHOD pattern.

Pattern Abuse

By this time you should be thinking, *"Is he serious? Does he really expect me to use this* `Application` *class for all new apps? It hasn't bought me anything, and it's overcomplicated the problem."*

I chose the example because it was simple and provided a good platform for showing the mechanics of TEMPLATE METHOD. On the other hand, I don't really recommend building `ftoc` like this.

This is a good example of pattern abuse. Using TEMPLATE METHOD for this particular application is ridiculous. It complicates the program and makes its bigger. Encapsulating the main loop of every application in the universe sounded wonderful when we started, but the practical application is fruitless in this case.

Design patterns are wonderful things. They can help you with many design problems. But the fact that they exist does not mean that they should always be used. In this case, while TEMPLATE METHOD was applicable to the problem, its use was not advisable. The cost of the pattern was higher than the benefit it yielded.

So let's look at a slightly more useful example. (See Listing 14-4.)

Bubble Sort[3]



Listing 14-4

BubbleSorter.java

```
public class BubbleSorter
{
  static int operations = 0;
  public static int sort(int [] array)
  {
    operations = 0;
    if (array.length <= 1)
      return operations;

    for (int nextToLast = array.length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
        compareAndSwap(array, index);

    return operations;
  }

  private static void swap(int[] array, int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  private static void compareAndSwap(int[] array, int index)
  {
    if (array[index] > array[index+1])
      swap(array, index);
    operations++;
  }
}
```

The `BubbleSorter` class knows how to sort an array of integers using the bubble-sort algorithm. The `sort` method of `BubbleSorter` contains the algorithm that knows how to do a bubble sort. The two ancillary methods, `swap` and `compareAndSwap`, deal with the details of integers and arrays and handle the mechanics that the `sort` algorithm requires.

3. Like `Application`, `Bubble Sort` is easy to understand and so makes a useful teaching tool. However, no one in their right mind would ever actually use a `Bubble Sort` if they had any significant amount of sorting to do. There are *much* better algorithms.

Using the TEMPLATE METHOD pattern, we can separate the bubble-sort algorithm out into an abstract base class named `BubbleSorter`. `BubbleSorter` contains an implementation of the `sort` function that calls an abstract method named `outOfOrder` and another called `swap`. The `outOfOrder` method compares two adjacent elements in the array and returns `true` if the elements are out of order. The `swap` method swaps two adjacent cells in the array.

The `sort` method does not know about the array, nor does it care what kind of objects are stored in the array. It just calls `outOfOrder` for various indices into the array and determines whether those indices should be swapped or not. (See Listing 14-5.)

Listing 14-5

BubbleSorter.java

```
public abstract class BubbleSorter
{
  private int operations = 0;
  protected int length = 0;

  protected int doSort()
  {
    operations = 0;
    if (length <= 1)
      return operations;

    for (int nextToLast = length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
      {
        if (outOfOrder(index))
          swap(index);
        operations++;
      }

    return operations;
  }

  protected abstract void swap(int index);
  protected abstract boolean outOfOrder(int index);
}
```

Given `BubbleSorter` we can now create simple derivatives that can sort any different kind of object. For example, we could create `IntBubbleSorter`, which sorts arrays of integers, and `DoubleBubbleSorter`, which sorts arrays of doubles. (See Figure 14-1, Listing 14-6, and Listing 14-7.)

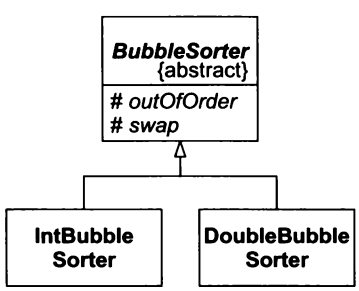


Figure 14-1 Bubble-Sorter Structure

Listing 14-6

IntBubbleSorter.java

```
public class IntBubbleSorter extends BubbleSorter
{
  private int[] array = null;
  public int sort(int [] theArray)
  {
    array = theArray;
    length = array.length;
    return doSort();
  }

  protected void swap(int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  protected boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

Listing 14-7

DoubleBubbleSorter.java

```
public class DoubleBubbleSorter extends BubbleSorter
{
  private double[] array = null;
  public int sort(double [] theArray)
  {
    array = theArray;
    length = array.length;
    return doSort();
  }

  protected void swap(int index)
  {
    double temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  protected boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

The TEMPLATE METHOD pattern shows one of the classic forms of reuse in object-oriented programming. Generic algorithms are placed in the base class and inherited into different detailed contexts. But this technique is not without its costs. Inheritance is a very strong relationship. Derivatives are inextricably bound to their base classes.

For example, the `outOfOrder` and `swap` functions of `IntBubbleSorter` are exactly what are needed for other kinds of sort algorithms. And yet, there is no way to reuse `outOfOrder` and `swap` in those other sort algorithms. By inheriting `BubbleSorter`, we have doomed `IntBubbleSorter` to be bound forever to `BubbleSorter`. The STRATEGY pattern provides another option.

STRATEGY

The STRATEGY pattern solves the problem of inverting the dependencies of the generic algorithm and the detailed implementation in a very different way. Consider, once again, the pattern-abusing `Application` problem.

Rather than placing the generic application algorithm into an abstract base class, we place it into a *concrete* class named `ApplicationRunner`. We define the abstract methods that the generic algorithm must call within an interface named `Application`. We derive `ftocStrategy` from this interface and pass it into the `ApplicationRunner`. `ApplicationRunner` then delegates to this interface. (See Figure 14-2, and Listings 14-8 through 14-10.)

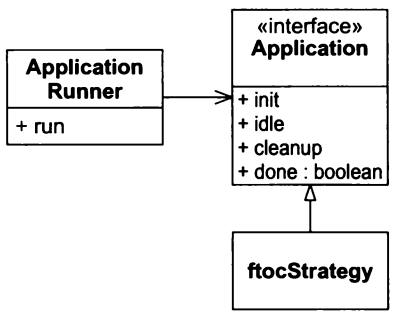


Figure 14-2 Strategy of the Application algorithm

Listing 14-8

ApplicationRunner.java

```
public class ApplicationRunner
{
  private Application itsApplication = null;

  public ApplicationRunner(Application app)
  {
    itsApplication = app;
  }
  public void run()
  {
    itsApplication.init();
    while (!itsApplication.done())
      itsApplication.idle();
    itsApplication.cleanup();
  }
}
```

Listing 14-9

Application.java

```
public interface Application
{
  public void init();
  public void idle();
```

```
  public void cleanup();
  public boolean done();
}
```

Listing 14-10

ftocStrategy.java

```
import java.io.*;
public class ftocStrategy implements Application
{
  private InputStreamReader isr;
  private BufferedReader br;
  private boolean isDone = false;

  public static void main(String[] args) throws Exception
  {
    (new ApplicationRunner(new ftocStrategy())).run();
  }

  public void init()
  {
    isr = new InputStreamReader(System.in);
    br = new BufferedReader(isr);
  }

  public void idle()
  {
    String fahrString = readLineAndReturnNullIfError();
    if (fahrString == null || fahrString.length() == 0)
      isDone = true;
    else
    {
      double fahr = Double.parseDouble(fahrString);
      double celcius = 5.0/9.0*(fahr-32);
      System.out.println("F=" + fahr + ", C=" + celcius);
    }
  }

  public void cleanup()
  {
    System.out.println("ftoc exit");
  }

  public boolean done()
  {
    return isDone;
  }

  private String readLineAndReturnNullIfError()
  {
    String s;
    try
    {
      s = br.readLine();
    }
    catch(IOException e)
```

```
    {
      s = null;
    }
    return s;
  }
}
```

It should be clear that this structure has both benefits and costs over the TEMPLATE METHOD structure. STRATEGY involves more total classes and more indirection than TEMPLATE METHOD. The delegation pointer within `ApplicationRunner` incurs a slightly higher cost in terms of run time and data space than inheritance would. On the other hand, if we had many different applications to run, we could reuse the `ApplicationRunner` *instance* and pass in many different implementations of `Application`, thereby reducing the coupling between the generic algorithm and the details it controls.

None of these costs and benefits are overriding. In most cases, none of them matters in the slightest. In the typical case, the most worrisome is the extra class needed by the STRATEGY pattern. However, there is more to consider.

Sorting Again

Consider implementating the bubble sort using the STRATEGY pattern. (See Listings 14-11 through 14-13.)

Listing 14-11

`BubbleSorter.java`

```
public class BubbleSorter
{
  private int operations = 0;
  private int length = 0;
  private SortHandle itsSortHandle = null;

  public BubbleSorter(SortHandle handle)
  {
    itsSortHandle = handle;
  }

  public int sort(Object array)
  {
    itsSortHandle.setArray(array);
    length = itsSortHandle.length();
    operations = 0;
    if (length <= 1)
      return operations;

    for (int nextToLast = length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
      {
        if (itsSortHandle.outOfOrder(index))
          itsSortHandle.swap(index);
        operations++;
      }

    return operations;
  }
}
```

Listing 14-12

SortHandle.java

```
public interface SortHandle
{
  public void swap(int index);
  public boolean outOfOrder(int index);
  public int length();
  public void setArray(Object array);
}
```

Listing 14-13

IntSortHandle.java

```
public class IntSortHandle implements SortHandle
{
  private int[] array = null;

  public void swap(int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  public void setArray(Object array)
  {
    this.array = (int[])array;
  }

  public int length()
  {
    return array.length;
  }

  public boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

Notice that the `IntSortHandle` class knows nothing of the `BubbleSorter`. It has no dependency whatever on the bubble-sort implementation. This is not the case with the TEMPLATE METHOD pattern. Look back at Listing 14-6, and you can see that the `IntBubbleSorter` depended directly on `BubbleSorter`, the class that contains the bubble-sort algorithm.

The TEMPLATE METHOD approach partially violates DIP by Implementating the `swap` and `outOfOrder` methods to depend directly on the bubble-sort algorithm. The STRATEGY approach contains no such dependency. Thus, we can use the `IntSortHandle` with `Sorter` implementations other than `BubbleSorter`.

For example, we can create a variation of the bubble sort that terminates early if a pass through the array finds it in order. (See Listing 14-14.) `QuickBubbleSorter` can also use `IntSortHandle` or any other class derived from `SortHandle`.

Listing 14-14

QuickBubbleSorter.java

```
public class QuickBubbleSorter
{
  private int operations = 0;
  private int length = 0;
  private SortHandle itsSortHandle = null;

  public QuickBubbleSorter(SortHandle handle)
  {
    itsSortHandle = handle;
  }

  public int sort(Object array)
  {
    itsSortHandle.setArray(array);
    length = itsSortHandle.length();
    operations = 0;
    if (length <= 1)
      return operations;

    boolean thisPassInOrder = false;
    for (int nextToLast = length-2; nextToLast >= 0 && !thisPassInOrder; nextToLast--)
    {
      thisPassInOrder = true; //potenially.
      for (int index = 0; index <= nextToLast; index++)
      {
        if (itsSortHandle.outOfOrder(index))
        {
          itsSortHandle.swap(index);
          thisPassInOrder = false;
        }
        operations++;
      }
    }

    return operations;
  }
}
```

Thus, the STRATEGY pattern provides one extra benefit over the TEMPLATE METHOD pattern. Whereas the TEMPLATE METHOD pattern allows a generic algorithm to manipulate many possible detailed implementations, the STRATEGY pattern by fully conforming to the DIP allows each detailed implementation to be manipulated by many different generic algorithms.

Conclusion

Both TEMPLATE METHOD and STRATEGY allow you to separate high-level algorithms from low-level details. Both allow the high-level algorithms to be reused independently of the details. At the cost of a little extra complexity, memory, and runtime, STRATEGY also allows the details to be reused independently of the high-level algorithm.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.

15

FACADE and MEDIATOR



Symbolism erects a facade of respectability to hide the indecency of dreams.

—Mason Cooley

The two patterns discussed in this chapter have a common purpose. Both impose some kind of policy on another group of objects. FACADE imposes policy from above, and MEDIATOR imposes policy from below. The use of FACADE is visible and constraining, while the use of MEDIATOR is invisible and enabling.

FACADE

The FACADE pattern is used when you want to provide a simple and specific interface onto a group of objects that has a complex and general interface. Consider, for example, `DB.java` in Listing 26-9 on page 333. This class imposes a very simple interface, specific to `ProductData`, on the complex and general interfaces of the classes within the `java.sql` package. Figure 15-1 shows the structure.

Notice that the DB class protects the `Application` from needing to know the intimacies of the `java.sql` package. It hides all the generality and complexity of `java.sql` behind a very simple and specific interface.

A FACADE like DB imposes a lot of policy on the usage of the `java.sql` package. It knows how to initialize and close the database connection. It knows how to translate the members of `ProductData` into database fields and back. It knows how to build the appropriate queries and commands to manipulate the database. And it hides all that complexity from its users. From the point of view of the `Application`, `java.sql` does not exist; it is hidden behind the FACADE.

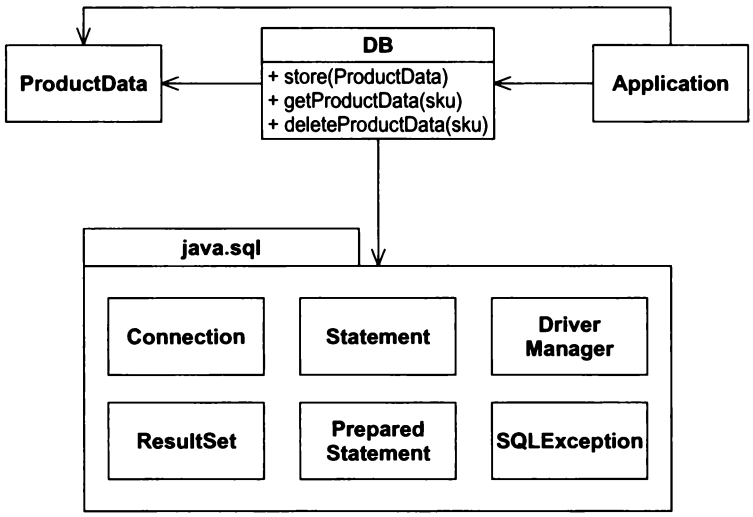


Figure 15-1 The DB FACADE

The use of the FACADE pattern implies that the developers have adopted the convention that all database calls must go through DB. If any part of the `Application` code goes straight to `java.sql` rather than through the FACADE, then that convention is violated. As such, the FACADE imposes its polices on the application. By convention, DB has become the sole broker of the facilities of `java.sql`.

MEDIATOR

The MEDIATOR pattern also imposes policy. However, whereas Facade imposed its policy in a visible and constraining way, MEDIATOR imposes its policies in a hidden and unconstrained way. For example, the `QuickEntryMediator` class in Listing 15–1 is a class that sits quietly behind the scenes and binds a text-entry field to a list. When you type in the text-entry field, the first element of the list that matches what you have typed is highlighted. This lets you type abbreviations and quickly select a list item.

Listing 15-1

`QuickEntryMediator.java`

```
package utility;

import javax.swing.*;
import javax.swing.event.*;

/**
QuickEntryMediator.  This class takes a JTextField and a JList.
It assumes that the user will type characters into the
JTextField that are prefixes of entries in the JList.  It
automatically selects the first item in the JList that matches
the current prefix in the JTextField.

If the JTextField is null, or the prefix does not match any
element in the JList, then the JList selection is cleared.

There are no methods to call for this object.  You simply
create it, and forget it.  (But don't let it be garbage
collected...)
```

Example:

```
JTextField t = new JTextField();
JList l = new JList();

QuickEntryMediator qem = new QuickEntryMediator(t, l);
 // that's all folks.

 @author Robert C. Martin, Robert S. Koss
 @date 30 Jun, 1999 2113 (SLAC)
 */

public class QuickEntryMediator {
  public QuickEntryMediator(JTextField t, JList l) {
    itsTextField = t;
    itsList = l;

    itsTextField.getDocument().addDocumentListener(
      new DocumentListener() {
        public void changedUpdate(DocumentEvent e) {
          textFieldChanged();
        }

        public void insertUpdate(DocumentEvent e) {
          textFieldChanged();
        }

        public void removeUpdate(DocumentEvent e) {
          textFieldChanged();
        }
      } // new DocumentListener
    ); // addDocumentListener
  } // QuickEntryMediator()

  private void textFieldChanged() {
    String prefix = itsTextField.getText();

    if (prefix.length() == 0) {
      itsList.clearSelection();
      return;
      }

      ListModel m = itsList.getModel();
      boolean found = false;
      for (int i = 0; found == false && i < m.getSize(); i++) {
      Object o = m.getElementAt(i);
      String s = o.toString();
      if (s.startsWith(prefix)) {
      itsList.setSelectedValue(o, true);
      found = true;
      }
    }

    if (!found) {
      itsList.clearSelection();
    }
  } // textFieldChanged
```

```
  private JTextField itsTextField;
  private JList itsList;
} // class QuickEntryMediator
```

The structure of the `QuickEntryMediator` is shown in Figure 15-2. An instance of `QuickEntryMediator` is constructed with a `JList` and a `JTextField`. The `QuickEntryMediator` registers an anonymous `DocumentListener` with the `JTextField`. This listener invokes the `textFieldChanged` method whenever there is a change in the text. This method then finds an element of the `JList` that is prefixed by the text and selects it.

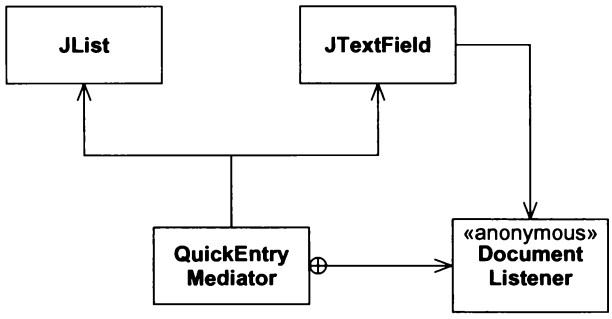


Figure 15-2 QuickEntryMediator

The users of the `JList` and `JTextField` have no idea that this MEDIATOR exists. It sits there quietly, imposing its policy on those objects without their permission or knowledge.

Conclusion

Imposing policy can be done from above using FACADE if that policy needs to be big and visible. On the other hand, if subtlety and discretion are needed, MEDIATOR may be the more appropriate choice. Facades are usually the focal point of a convention. Everyone agrees to use the facade instead of the objects beneath it. Mediator, on the other hand, is hidden from the users. Its policy is a fait accompli rather than a matter of convention.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

16

SINGLETON and MONOSTATE



"Infinite beatitude of existence! It is; and there is none else beside It."

—The point. Flatland. Edwin A. Abbott

Usually there is a one-to-many relationship between classes and instances. You can create many instances of most classes. The instances are created when they are needed and disposed of when their usefulness ends. They come and go in a flow of memory allocations and deallocations.

However, there are some classes that should have only one instance. That instance should appear to have come into existence when the program started and should be disposed of only when the program ends. Such objects are sometimes the roots of the application. From the roots you can find your way to many other objects in the system. Sometimes they are factories, which you can use to create the other objects in the system. Sometimes these objects are managers, responsible for keeping track of certain other objects and driving them through their paces.

Whatever these objects are, it is a severe logic failure if more than one of them are created. If more than one root are created, then access to objects in the application may depend on a chosen root. Programmers, not knowing that more than one root exist, may find themselves looking at a subset of the application objects without knowing it. If more than one factory exist, clerical control over the created objects may be compromised. If more than one manager exist, activities that were intended to be serial may become concurrent.

It may seem that mechanisms to enforce the singularity of these objects are overkill. After all, when you initialize the application, you can simply create one of each and be done with it. In fact, this is usually the best course of action. Mechanism should be avoided when there is no immediate and significant need. However, we also want our code to communicate our intent. If the mechanism for enforcing singularity is trivial, the benefit of communication may outweigh the cost of the mechanism.

This chapter is about two patterns that enforce singularity. These patterns have very different cost–benefit trade-offs. In many contexts, their cost is low enough to more than balance the benefit of their expressiveness.

SINGLETON[1]

SINGLETON is a very simple pattern. The test case in Listing 16-1 shows how it should work. The first test function shows that the `Singleton` instance is accessed through the public static method `Instance`. It also shows that if `Instance` is called multiple times, a reference to the exact same instance is returned each time. The second test case shows that the `Singleton` class has no public constructors, so there is no way for anyone to create an instance without using the `Instance` method.

Listing 16-1

Singleton Test Case

```
import junit.framework.*;
import java.lang.reflect.Constructor;

public class TestSimpleSingleton extends TestCase
{
  public TestSimpleSingleton(String name)
  {
    super(name);
  }

  public void testCreateSingleton()
  {
    Singleton s = Singleton.Instance();
    Singleton s2 = Singleton.Instance();
    assertSame(s, s2);
  }

  public void testNoPublicConstructors() throws Exception
  {
    Class singleton = Class.forName("Singleton");
    Constructor[] constructors = singleton.getConstructors();
    assertEquals("public constructors.",
                 0, constructors.length);
  }
}
```

This test case is a specification for the SINGLETON pattern. It leads directly to the code shown in Listing 16-2. It should be clear, by inspecting this code, that there can never be more than one instance of the `Singleton` class within the scope of the static variable `Singleton.theInstance`.

1. [GOF95], p. 127.

Listing 16-2

Singleton Implementation

```
public class Singleton
{
  private static Singleton theInstance = null;
  private Singleton() {}

  public static Singleton Instance()
  {
    if (theInstance == null)
      theInstance = new Singleton();
    return theInstance;
  }
}
```

Benefits of the SINGLETON

- **Cross platform.** Using appropriate middleware (e.g., RMI), SINGLETON can be extended to work across many JVMs and many computers.
- **Applicable to any class.** You can change any class into a SINGLETON simply by making its constructors private and by adding the appropriate static functions and variable.
- **Can be created through derivation.** Given a class, you can create a subclass that is a SINGLETON.
- **Lazy evaluation**. If the SINGLETON is never used, it is never created.

Costs of the SINGLETON

- **Destruction is undefined.** There is no good way to destroy or decommission a SINGLETON. If you add a `decommission` method that nulls out `theInstance`, other modules in the system may still be holding a reference to the SINGLETON instance. Subsequent calls to `Instance` will cause another instance to be created, causing two concurrent instances to exist. This problem is particularly acute in C++ where the instance *can be destroyed*, leading to possible dereferencing of a destroyed object.
- **Not inherited.** A class derived from a SINGLETON is not a singleton. If it needs to be a SINGLETON, the static function and variable need to be added to it.
- **Efficiency.** Each call to `Instance` invokes the `if` statement. For most of those calls, the `if` statement is useless.
- **Nontransparent.** Users of a SINGLETON know that they are using a SINGLETON because they must invoke the `Instance` method.

SINGLETON in Action

Assume that we have a Web-based system that allows users to log in to secure areas of a Web server. Such a system will have a database containing user names, passwords, and other user attributes. Assume further that the database is accessed through a third-party API. We could access the database directly in every module that needed to read and write a user. However, this would scatter usage of the third-party API throughout the code, and it would leave us no place to enforce access or structure conventions.

A better solution is to use the FACADE pattern and create a `UserDatabase` class that provides methods for reading and writing `User` objects. These methods access the third-party API of the database, translating between `User` objects and the tables and rows of the database. Within the `UserDatabase`, we can enforce conventions of structure and access. For example, we can guarantee that no `User` record gets written unless it has a nonblank `username`. Or we can serialize access to a `User` record, making sure that two modules cannot simultaneously read and write it.

The code in Listings 16-3 and 16-4 shows a SINGLETON solution. The SINGLETON class is named `UserDatabaseSource`. It implements the `UserDatabase` interface. Notice that the static `instance()` method does not have the traditional `if` statement to protect against multiple creations. Instead, it takes advantage of the Java initialization facility.

Listing 16-3

`UserDatabase Interface`

```
public interface UserDatabase
{
  User readUser(String userName);
  void writeUser(User user);
}
```

Listing 16-4

`UserDatabaseSource Singleton`

```
public class UserDatabaseSource implements UserDatabase
{
  private static UserDatabase theInstance =
    new UserDatabaseSource();

  public static UserDatabase instance()
  {
    return theInstance;
  }

  private UserDatabaseSource()
  {
  }

  public User readUser(String userName)
  {
    // Some Implementation
    return null;  // just to make it compile.
  }

  public void writeUser(User user)
  {
    // Some Implementation
  }
}
```

This is an extremely common use of the SINGLETON pattern. It assures that all database access will be through a single instance of `UserDatabaseSource`. This makes it easy to put checks, counters, and locks in `UserDatabaseSource` that enforce the access and structure conventions mentioned earlier.

MONOSTATE[2]

The MONOSTATE pattern is another way to achieve singularity. It works through a completely different mechanism. We can see how that mechanism works by studying the MONOSTATE test case in Listing 16-5.

The first test function simply describes an object whose `x` variable can be set and retrieved. But the second test case shows that two instances of the same class behave *as though they were one*. If you set the `x` variable on

2. [BALL2000].

one instance to a particular value, you can retrieve that value by getting the x variable of a different instance. It's as though the two instances are just different names for the same object.

Listing 16-5

Monostate Test Case

```
import junit.framework.*;

public class TestMonostate extends TestCase
{
  public TestMonostate(String name)
  {
    super(name);
  }

  public void testInstance()
  {
    Monostate m = new Monostate();
    for (int x = 0; x<10; x++)
    {
      m.setX(x);
      assertEquals(x, m.getX());
    }
  }

  public void testInstancesBehaveAsOne()
  {
    Monostate m1 = new Monostate();
    Monostate m2 = new Monostate();

    for (int x = 0; x<10; x++)
    {
      m1.setX(x);
      assertEquals(x, m2.getX());
    }
  }
}
```

If we were to plug the `Singleton` class into this test case and replace all the `new Monostate` statements with calls to `Singleton.Instance`, the test case should still pass. So this test case describes the *behavior* of `Singleton` without imposing the constraint of a single instance!

How can two instances behave as though they were a single object? Quite simply it means that the two objects must share the same variables. This is easily achieved by making all the variables static. Listing 16-6 shows the implementation of `Monostate` that passes the above test case. Note that the `itsX` variable is static. Note also that *none of the methods are static*. This is important as we'll see later.

Listing 16-6

Monostate Implementation

```
public class Monostate
{
  private static int itsX = 0;
  public Monostate() {}

  public void setX(int x)
```

```
  {
    itsX = x;
  }

  public int getX()
  {
    return itsX;
  }
}
```

I find this to be a delightfully twisted pattern. No matter how many instances of `Monostate` you create, they all behave as though they were a *single object*. You can even destroy or decommission all the current instances without losing the data.

Note that the difference between the two patterns is one of behavior vs. structure. The SINGLETON pattern enforces the structure of singularity. It prevents any more than one instance from being created. Whereas MONOSTATE enforces the *behavior* of singularity without imposing structural constraints. To underscore this difference consider that the MONOSTATE test case is valid for the `Singleton` class, but the SINGLETON test case is not even close to being valid for the `Monostate` class.

Benefits of MONOSTATE

- **Transparency**. Users of a MONOSTATE do not behave differently than users of a regular object. The users do not need to know that the object is MONOSTATE.
- **Derivability**. Derivatives of a MONOSTATE are MONOSTATES. Indeed, all the derivatives of a MONOSTATE are part of the *same* MONOSTATE. They all share the same static variables.
- **Polymorphism**. Since the methods of a MONOSTATE are not static, they can be overridden in a derivative. Thus different derivatives can offer different behavior over the same set of static variables.
- **Well-defined creation and destruction.** The variables of a MONOSTATE, being static, have well-defined creation and destruction times.

Costs of MONOSTATE

- **No conversion**. A normal class cannot be converted into a MONOSTATE class through derivation.
- **Efficiency**. A MONOSTATE may go through many creations and destructions because it is a real object. These operations are often costly.
- **Presence**. The variables of a MONOSTATE take up space, even if the MONOSTATE is never used.
- **Platform local.** You can't make a MONOSTATE work across several JVM instances or across several platforms.

MONOSTATE in Action

Consider implementing the simple finite state machine for a subway turnstile shown in Figure 16-1. The turnstile begins its life in the `Locked` state. If a coin is deposited, it transitions to the `Unlocked` state, unlocks the gate, resets any alarm state that might be present, and deposits the coin in its collection bin. If a user passes through the gate at this point, the turnstile transitions back to the `Locked` state and locks the gate.

There are two abnormal conditions. If the user deposits two or more coins before passing through the gate, the coins will be refunded and the gate will remain unlocked. If the user passes through without paying, then an alarm will sound and the gate will remain locked.

The test program that describes this operation is shown in Listing 16-7. Note that the test methods assume that the `Turnstile` is a monostate. It expects to be able to send events and gather queries from different instances. This makes sense if there will never be more than one instance of the `Turnstile`.

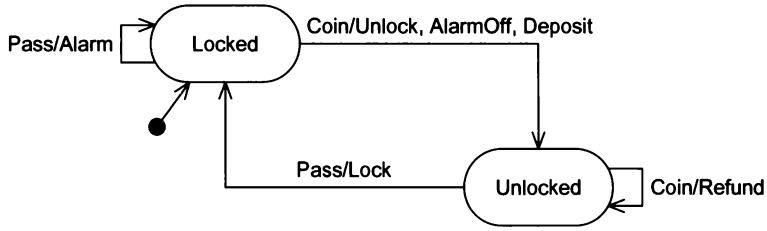


Figure 16-1 Subway Turnstile Finite State Machine

Listing 16-7

`TestTurnstile`

```
import junit.framework.*;

public class TestTurnstile extends TestCase
{
  public TestTurnstile(String name)
  {
    super(name);
  }

  public void setUp()
  {
    Turnstile t = new Turnstile();
    t.reset();
  }

  public void testInit()
  {
    Turnstile t = new Turnstile();
    assert(t.locked());
    assert(!t.alarm());
  }

  public void testCoin()
  {
    Turnstile t = new Turnstile();
    t.coin();
    Turnstile t1 = new Turnstile();
    assert(!t1.locked());
    assert(!t1.alarm());
    assertEquals(1, t1.coins());
  }

  public void testCoinAndPass()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.pass();

    Turnstile t1 = new Turnstile();
    assert(t1.locked());
    assert(!t1.alarm());
    assertEquals("coins", 1, t1.coins());
  }
```

```
  public void testTwoCoins()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.coin();

    Turnstile t1 = new Turnstile();
    assert("unlocked", !t1.locked());
    assertEquals("coins",1, t1.coins());
    assertEquals("refunds", 1, t1.refunds());
    assert(!t1.alarm());
  }

  public void testPass()
  {
    Turnstile t = new Turnstile();
    t.pass();
    Turnstile t1 = new Turnstile();
    assert("alarm", t1.alarm());
    assert("locked", t1.locked());
  }

  public void testCancelAlarm()
  {
    Turnstile t = new Turnstile();
    t.pass();
    t.coin();
    Turnstile t1 = new Turnstile();
    assert("alarm", !t1.alarm());
    assert("locked", !t1.locked());
    assertEquals("coin", 1, t1.coins());
    assertEquals("refund", 0, t1.refunds());
  }

  public void testTwoOperations()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.pass();
    t.coin();
    assert("unlocked", !t.locked());
    assertEquals("coins", 2, t.coins());
    t.pass();
    assert("locked", t.locked());
  }
}
```

The implementation of the monostate `Turnstile` is in Listing 16-8. The base `Turnstile` class delegates the two event functions (`coin` and `pass`) to two derivatives of `Turnstile` (`Locked` and `Unlocked`) that represent the states of the finite-state machine.

Listing 16-8

Turnstile

```
public class Turnstile
{
  private static boolean isLocked = true;
  private static boolean isAlarming = false;
  private static int itsCoins = 0;
  private static int itsRefunds = 0;
  protected final static Turnstile LOCKED = new Locked();
  protected final static Turnstile UNLOCKED = new Unlocked();
  protected static Turnstile itsState = LOCKED;

  public void reset()
  {
    lock(true);
    alarm(false);
    itsCoins = 0;
    itsRefunds = 0;
    itsState = LOCKED;
  }

  public boolean locked()
  {
    return isLocked;
  }

  public boolean alarm()
  {
    return isAlarming;
  }

  public void coin()
  {
    itsState.coin();
  }

  public void pass()
  {
    itsState.pass();
  }

  protected void lock(boolean shouldLock)
  {
    isLocked = shouldLock;
  }

  protected void alarm(boolean shouldAlarm)
  {
    isAlarming = shouldAlarm;
  }

  public int coins()
  {
    return itsCoins;
  }
```